

Text Analytics 2026 — Assignment 1

N-gram Language Models με Laplace και Kneser–Ney Smoothing

Text Analytics 2026 — Assignment 1

N-gram Language Models με Laplace και Kneser–Ney Smoothing

Φοιτητές: Νέγκι Χότζα — Γιώργος Κωστόπουλος

Colab notebook: [link](#)

Αρχείο notebook: Assignment1_Text_Analytics.ipynb

1. Σκοπός της εργασίας

Σκοπός της εργασίας ήταν η υλοποίηση και αξιολόγηση n-gram language models πάνω σε πραγματικό corpus κειμένων. Συγκεκριμένα, υλοποιήθηκαν bigram και trigram μοντέλα, με δύο διαφορετικές τεχνικές smoothing: Laplace/add-one smoothing και Kneser–Ney smoothing. Επιπλέον, τα μοντέλα χρησιμοποιήθηκαν για autocomplete, ώστε να συγκριθεί όχι μόνο η αριθμητική τους επίδοση αλλά και η ποιότητα των παραγόμενων προτάσεων.

Ένα n-gram language model προσεγγίζει την πιθανότητα ολόκληρης ακολουθίας με γινόμενο πιθανοτήτων υπό περιορισμένο context. Για ένα bigram μοντέλο:

$$P(w_1, \dots, w_N) \approx \prod_{i=1}^N P(w_i | w_{i-1})$$

και για ένα trigram μοντέλο:

$$P(w_1, \dots, w_N) \approx \prod_{i=1}^N P(w_i | w_{i-2}, w_{i-1})$$

Για αριθμητική σταθερότητα δεν υπολογίζουμε γινόμενα μικρών πιθανοτήτων, αλλά αθροίσματα λογαρίθμων:

$$\log_2 P(w_1, \dots, w_N) = \sum_{i=1}^N \log_2 P(w_i | \text{context}_i)$$

2. Δεδομένα και προεπεξεργασία

Χρησιμοποιήθηκε το Brown corpus από το NLTK και συγκεκριμένα η κατηγορία lore, η οποία περιέχει 4.881 προτάσεις. Οι προτάσεις χωρίστηκαν τυχαία σε training, development και test set με αναλογία 80% / 10% / 10%, δηλαδή 3.904 προτάσεις για εκπαίδευση, 488 για development και 489 για test.

Τα βασικά preprocessing βήματα ήταν τα εξής:

- μετατροπή όλων των tokens σε πεζά, ώστε λέξεις όπως “The” και “the” να αντιμετωπίζονται ως η ίδια λέξη,
- τυχαίο shuffle των προτάσεων με σταθερό seed για αναπαραγωγικότητα,
- δημιουργία vocabulary μόνο από το training set, ώστε να μην υπάρχει πληροφορία από development ή test κατά την εκπαίδευση,
- διατήρηση στο vocabulary μόνο των λέξεων με συχνότητα τουλάχιστον 10 στο training set,
- αντικατάσταση όλων των υπόλοιπων λέξεων με το ειδικό token *UNK*,
- προσθήκη των ειδικών tokens *start*, *start1*, *start2* και *end* για την αρχή και το τέλος των προτάσεων.

Πιο φορμαλιστικά, το vocabulary V ορίζεται ως:

$$V = \{w: \text{count}_{\text{train}}(w) \geq 10\} \cup \{ * UNK *, * end * \}$$

και η αντικατάσταση OOV λέξεων είναι:

$$\text{replace}(w) = \begin{cases} w, & \text{αν } w \in V \\ * UNK *, & \text{αλλιώς} \end{cases}$$

Το τελικό vocabulary είχε μέγεθος 966 tokens. Μετά την αντικατάσταση OOV λέξεων, μετρήθηκαν 23.237 εμφανίσεις του *UNK* στο training set, 2.892 στο development set και 2.981 στο test set.

3. Υλοποίηση των language models

Η υλοποίηση βασίστηκε σε μία κοινή κλάση NGramLM, η οποία περιέχει τη γενική λογική των n-gram μοντέλων: αποθήκευση της τιμής n , vocabulary, n-gram counts, context counts, προσθήκη start/end tokens και υπολογισμό log-probability. Πάνω σε αυτήν την κοινή βάση υλοποιήθηκαν δύο υποκλάσεις: LaplaceLM και KneserNeyLM. Με αυτόν τον τρόπο, η βασική n-gram λογική παραμένει κοινή και αλλάζει μόνο ο τρόπος υπολογισμού της πιθανότητας $P(w \mid \text{context})$.

Κλάση	Ρόλος	Περιγραφή
NGramLM	Βασική κλάση	Κοινή λογική για counts, contexts, start/end tokens και log-probability.
LaplaceLM	Υποκλάση	Υπολογίζει πιθανότητες με add-one/Laplace

	smoothing	smoothing.
KneserNeyLM	Υποκλάση smoothing	Υπολογίζει πιθανότητες με Kneser–Ney smoothing και backoff.

4. Smoothing methods

4.1 Μέγιστη πιθανοφάνεια (MLE)

Για ένα n-gram με context h , η μέγιστη πιθανοφάνεια θα ήταν:

$$P_{MLE}(w | h) = \frac{\text{count}(h, w)}{\text{count}(h)}$$

Δεν τη χρησιμοποιούμε άμεσα, γιατί δίνει πιθανότητα μηδέν σε κάθε άγνωστο n-gram, κάτι που μηδενίζει την πιθανότητα ολόκληρου του test corpus και κάνει το perplexity να εκραγεί.

4.2 Laplace (add-one) smoothing

Το Laplace smoothing προσθέτει 1 σε κάθε πιθανό n-gram:

$$P_{Laplace}(w | h) = \frac{\text{count}(h, w) + 1}{\text{count}(h) + |V|}$$

Πιο γενικά, με add- α smoothing:

$$P(w | h) = \frac{\text{count}(h, w) + \alpha}{\text{count}(h) + \alpha \cdot |V|}$$

Στην εργασία χρησιμοποιείται καθαρό add-one smoothing, δηλαδή $\alpha = 1$. Έτσι αποφεύγονται μηδενικές πιθανότητες στο test set, αλλά η μέθοδος μοιράζει αρκετή πιθανότητα και σε σπάνια ή μη εμφανισμένα n-grams, κάτι που μπορεί να αυξήσει το perplexity.

4.3 Kneser–Ney smoothing

Το Kneser–Ney smoothing είναι πιο ισχυρή μέθοδος, επειδή δεν βασίζεται μόνο στη συχνότητα μιας λέξης αλλά και στο σε πόσα διαφορετικά contexts εμφανίζεται.

Bigram Kneser–Ney. Για bigram model με context h :

$$P_{KN}(w | h) = \frac{\max(\text{count}(h, w) - D, 0)}{\text{count}(h)} + \lambda(h) \cdot P_{cont}(w)$$

όπου το **backoff weight** είναι:

$$\lambda(h) = \frac{D \cdot N_{1+}(h, \bullet)}{\text{count}(h)}$$

και η **continuation probability** είναι:

$$P_{cont}(w) = \frac{N_{1+}(\bullet, w)}{N_{1+}(\bullet, \bullet)}$$

όπου:

- D είναι το **discount** που αφαιρείται από κάθε count,
- $N_{1+}(h, \bullet)$: πόσες διαφορετικές λέξεις έχουν εμφανιστεί μετά το context h ,
- $N_{1+}(\bullet, w)$: σε πόσα διαφορετικά προηγούμενα contexts έχει εμφανιστεί η λέξη w ,
- $N_{1+}(\bullet, \bullet)$: ο συνολικός αριθμός διαφορετικών bigrams στο training set.

Trigram Kneser–Ney. Για trigram model χρησιμοποιούμε interpolated backoff στο bigram KN μοντέλο:

$$P_{KN}(w | u, v) = \frac{\max(\text{count}(u, v, w) - D, 0)}{\text{count}(u, v)} + \lambda(u, v) \cdot P_{KN}(w | v)$$

όπου:

$$\lambda(u, v) = \frac{D \cdot N_{1+}(u, v, \bullet)}{\text{count}(u, v)}$$

Το discount D επιλέχθηκε στο development set, πάνω από τις τιμές $D \in \{0.25, 0.5, 0.75, 0.9\}$:

Μοντέλο	Τιμές D που δοκιμάστηκαν	Καλύτερο D	Dev perplexity
Kneser–Ney Bigram	0.25, 0.5, 0.75, 0.9	0.75	36.229
Kneser–Ney Trigram	0.25, 0.5, 0.75, 0.9	0.9	36.498

5. Αξιολόγηση μοντέλων

Η αξιολόγηση έγινε με **cross-entropy** και **perplexity** στο test set.

Το cross-entropy ορίζεται ως η μέση αρνητική log πιθανότητα ανά token:

$$H = -\frac{1}{N} \sum_{i=1}^N \log_2 P(w_i | \text{context}_i)$$

και το perplexity είναι:

$$PP = 2^H$$

Το N είναι ο αριθμός των tokens που προβλέπουμε. Δεν προβλέπουμε ούτε μετράμε τα start pseudo-tokens, αλλά μετράμε κανονικά το *end*. Μικρότερες τιμές perplexity σημαίνουν καλύτερο language model, επειδή το μοντέλο θεωρεί πιο πιθανές τις πραγματικές προτάσεις του test set.

Smoothing	Model	Cross-entropy	Perplexity
Laplace	Bigram	6.0014	64.0617
Laplace	Trigram	7.4798	178.5000
Kneser–Ney	Bigram	5.1382	35.2165
Kneser–Ney	Trigram	5.1404	35.2698

Από τα αποτελέσματα φαίνεται ότι τα Kneser–Ney μοντέλα έχουν πολύ χαμηλότερο perplexity από τα Laplace μοντέλα. Το Laplace trigram έχει τη χειρότερη επίδοση, κάτι που δείχνει ότι το απλό add-one smoothing δεν είναι κατάλληλο όταν ο χώρος των πιθανών trigrams είναι πολύ μεγάλος και sparse. Τα Kneser–Ney bigram και trigram έχουν σχεδόν ίδια test perplexity, με το bigram να είναι ελάχιστα καλύτερο αριθμητικά στο συγκεκριμένο split.

6. Autocomplete και decoding

Πέρα από την αριθμητική αξιολόγηση, τα μοντέλα χρησιμοποιήθηκαν για autocomplete. Δοκιμάστηκαν 2 στρατηγικές decoding: greedy decoding, beam search.

6.1 Greedy decoding

Σε κάθε βήμα επιλέγουμε την πιο πιθανή επόμενη λέξη:

$$\hat{w} = \operatorname{argmax}_{w \in V} P(w \mid \text{context})$$

6.2 Beam search

Διατηρούμε τις B καλύτερες μερικές συνέχειες (beams) σε κάθε βήμα. Κάθε beam s έχει σωρευτικό score:

$$\text{score}(s) = \sum_i \log_2 P(w_i \mid \text{context}_i)$$

Επειδή το beam search προτιμά μικρότερες προτάσεις, χρησιμοποιείται **length normalization** με εκθέτη 0.7:

$$\text{score}_{\text{norm}}(s) = \frac{\sum_i \log_2 P(w_i \mid \text{context}_i)}{L^{0.7}}$$

όπου L είναι το πλήθος των παραγόμενων λέξεων (όχι του prompt).

6.3 Επιλογή υπερπαραμέτρων με grid search

Για τις μεθόδους με υπερπαραμέτρους έγινε grid search στο development set. Το καλύτερο setting επιλέχθηκε με heuristic score που συνδυάζει log πιθανότητα, ολοκλήρωση με *end*, αποφυγή επαναλήψεων και ποικιλία μέσω distinct-2:

$$\text{score} = \overline{\log_2 P} + 1.5 \cdot \text{ended} + 0.8 \cdot \text{distinct}_2 - 0.5 \cdot \text{repeated}_{3\text{-gram}} - 0.03 \cdot |\text{new_tokens} - 12|$$

όπου $\overline{\log_2 P}$ είναι το μέσο log probability των generated tokens, και τα υπόλοιπα είναι αυτόματα metrics που μετράμε ανά completion.

Model	Best config	Score	GenPPL	EndRate	Dist2
Laplace Bigram	beam_w8	-2.616	27.67	1.00	1.000
Laplace Trigram	beam_w5	-6.061	335.62	0.80	0.945
KN Bigram	beam_w8	-0.746	7.27	1.00	1.000
KN Trigram	beam_w5	-0.041	4.57	1.00	1.000

Το καλύτερο decoding configuration για τα δύο Kneser–Ney μοντέλα είχε αισθητά καλύτερο score και χαμηλότερο generated perplexity σε σχέση με τα Laplace μοντέλα. Ειδικά το KN Trigram με beam_w5 πέτυχε το καλύτερο συνολικό score και end rate 1.00 στο tuning.

Το **generated perplexity** που εμφανίζεται στον πίνακα υπολογίζεται μόνο πάνω στις παραγόμενες λέξεις και δίνεται από:

$$\text{GenPPL} = 2^{-\overline{\log_2 P}}$$

7. Ενδεικτικά autocomplete examples

Καλύτερα examples που μπορούν να παρουσιαστούν στο report:

- KN Trigram / beam_w5: Prompt “in the” → “in the united states . end”
- KN Trigram / beam_w5: Prompt “she had” → “she had been in the united states . end”
- KN Trigram / beam_w5: Prompt “the old” → “the old school . end”
- KN Bigram / beam_w8: Prompt “the old” → “the old red river . end”

Ένα προβληματικό example που δείχνει αδυναμία του Laplace bigram:

- Laplace Bigram / beam_w8: Prompt “she had” → “she had been in the most of the `` i have to be a `` i was to the first , and”

Το προβληματικό παράδειγμα δεν ολοκληρώνεται με `*end*` και η φράση δεν είναι φυσική. Αυτό δείχνει ότι το bigram context, σε συνδυασμό με Laplace smoothing, μπορεί να οδηγήσει σε τοπικά συχνές αλλά μη συνεκτικές ακολουθίες.

8. Συμπεράσματα

- Το Kneser–Ney smoothing έδωσε σαφώς καλύτερα αποτελέσματα από το Laplace smoothing ως προς το perplexity.
- Το Laplace trigram είχε ιδιαίτερα υψηλό perplexity, λόγω sparsity στα trigrams και υπερβολικά απλής ανακατανομής πιθανότητας.
- Τα trigram μοντέλα παράγουν συνήθως πιο φυσικές προτάσεις από τα bigram μοντέλα, επειδή χρησιμοποιούν μεγαλύτερο context.
- Το perplexity είναι κατάλληλο για σύγκριση language models, ενώ τα autocomplete examples δείχνουν πρακτικά πώς συμπεριφέρεται το μοντέλο στην παραγωγή κειμένου.
- Το tuned KN Trigram ήταν το πιο ισορροπημένο μοντέλο για autocomplete, επειδή συνδύασε χαμηλό generated perplexity, ολοκλήρωση προτάσεων και απουσία επαναλήψεων.

9. Συνεισφορά ομάδας

Η εργασία υλοποιήθηκε συνεργατικά από τους δύο φοιτητές. Η υλοποίηση περιλαμβάνει την προεπεξεργασία των δεδομένων, τη δημιουργία vocabulary, την υλοποίηση των n-gram counts, τα δύο smoothing methods, την αξιολόγηση με cross-entropy/perplexity και το autocomplete με διαφορετικές decoding στρατηγικές. Το τελικό notebook περιέχει τόσο τον κώδικα όσο και επεξηγηματικά markdown cells, ώστε να είναι αναπαραγώγιμη η διαδικασία και κατανοητά τα αποτελέσματα.